

AI-Powered Handwriting Recognition & Smart Document Processing Platform

MERN Stack + Python ML Service | Web App Only | Built with Antigravity AI Agent

React.js · Redux Toolkit · Tailwind CSS · Node.js · Express · Python FastAPI · MongoDB

AI/ML/DL

MERN Stack

Web App

MongoDB

REST API

TABLE OF CONTENTS

#	Section	Page
1	Project Overview & Objectives	2
2	System Architecture Diagram	3
3	ML/AI Processing Pipeline	4
4	User Workflow (End-to-End)	5
5	Database Design (MongoDB Collections)	6
6	API Endpoint Architecture	7
7	Technology Stack	8
8	Module-wise Implementation Plan	9
9	Versioned Roadmap (V1→V3)	10
10	Deployment & DevOps Pipeline	11
11	Security Architecture	12
12	Testing Strategy	12
13	Risks & Mitigations	13
14	Building This With Antigravity	14

1. Project Overview & Objectives

This project proposes an **AI-powered Handwriting Recognition and Smart Document Processing Platform** delivered as a single responsive **web application** — no native mobile app is built. It converts handwritten text from uploaded images, scanned documents, and real-time browser camera capture into structured, searchable, and editable digital content. The system leverages Deep Learning models including **TrOCR** (Transformer-based OCR), **CRNN+CTC**, and the **EAST text detector**, served through a small dedicated Python ML service while the rest of the application runs on the **MERN stack** (MongoDB, Express, React, Node.js).

Core Objectives

Icon	Objective	Description
■	Accurate Recognition	Achieve 90%+ character recognition accuracy on clean handwritten text using fine-tuned TrOCR.
■	Web-Only Platform	Single responsive React.js web app — works on desktop and mobile browsers, including camera capture.
■	Real-time Processing	Process uploaded images within 3–5 seconds for standard A4 documents.
■	Security First	JWT-based authentication, bcrypt password hashing, and input validation across all endpoints.
■	Export Flexibility	Export recognized text as TXT (V1), then DOCX, PDF with confidence highlighting (V2).
■	Minimal-Footprint Architecture	One Node/Express backend for everything except ML; one small Python service strictly for OCR inference.
■	Continuity-Friendly Build Process	Every implementation step is logged so the build can pause and resume across different AI agent sessions.

Problem Statement

Despite digitization, enormous amounts of information remain trapped in handwritten form — student notes, medical prescriptions, legal documents, historical records, and field reports. Existing solutions like Google Lens offer basic OCR but lack structured output, confidence scoring, annotation tools, domain-specific tuning, or proper history and export capabilities. This platform bridges that gap with a focused, web-first approach built on a familiar, fast-to-ship MERN stack.

Target Users

User Segment	Use Case	Key Feature Needed
Students & Educators	Digitize handwritten notes, assignments	Batch upload, export to DOCX/PDF
Healthcare	Prescription, patient notes recognition	Domain model fine-tuning (future scope)
Legal / Office Staff	Form and note digitization	High accuracy + history/search
Researchers	Historical document transcription	Old handwriting support (future scope)
Anyone on the go	Browser camera capture from a phone	Mobile-responsive web UI (no app install)

2. System Architecture

The platform uses a deliberately simple two-service backend: a **Node.js + Express** service that handles everything except ML (auth, document metadata, history, export, search), and a small **Python + FastAPI** service used strictly for OCR/ML inference. Both connect to the same **MongoDB** database, viewable in development through **MongoDB Compass**. There is no separate API gateway, message queue, or cache layer in V1 — those are deferred until they're actually needed, to keep the build manageable for a solo/small-team project.

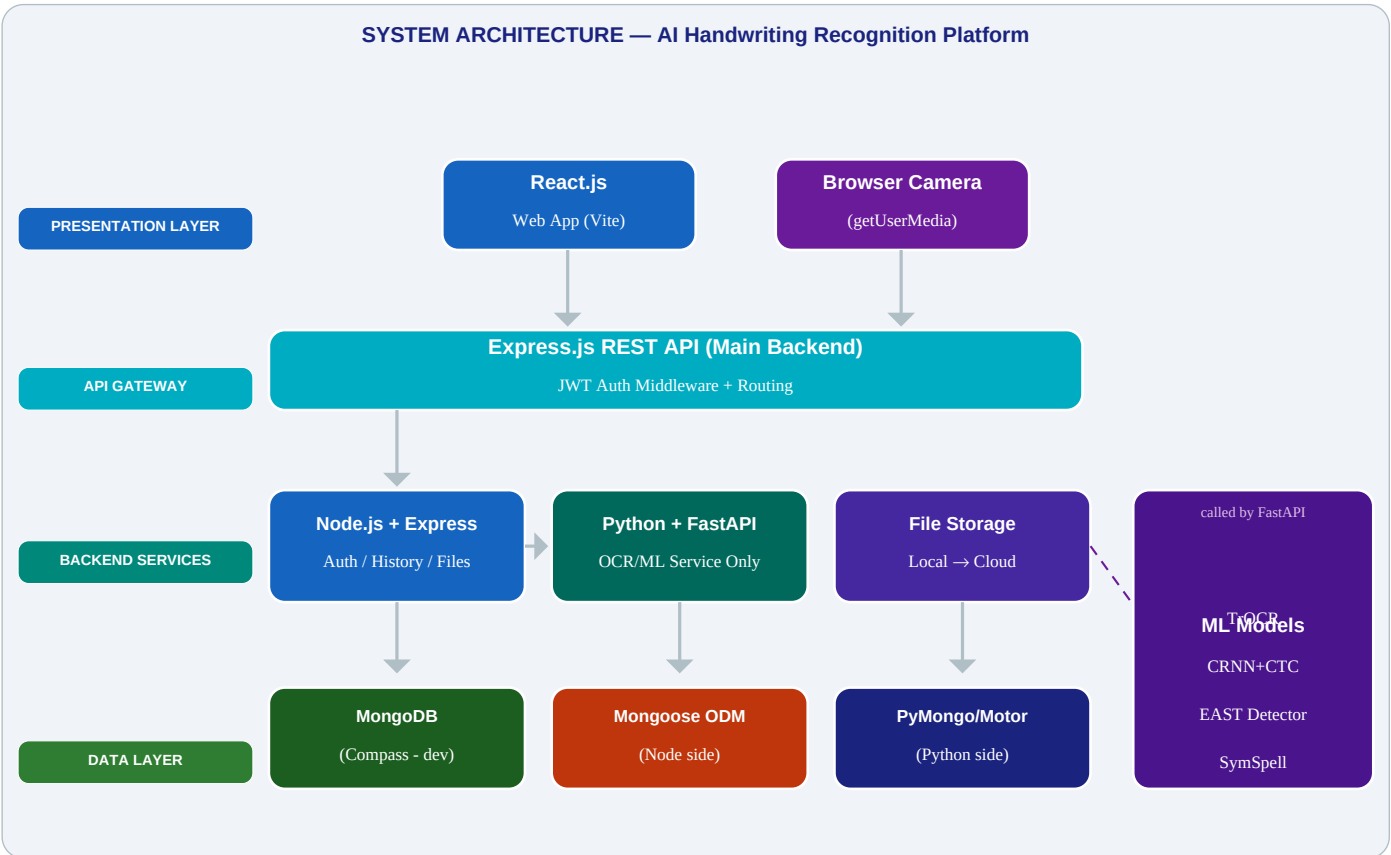


Figure 1 — System Architecture: React web client, Express main backend, Python ML service, shared MongoDB

Architecture Components

Layer	Component	Technology	Responsibility
Presentation	Web App	React.js + Redux Toolkit + Tailwind CSS	User interface, file upload, browser camera, text editor
Backend	Main API	Node.js + Express + JWT	Auth, document metadata, history, export, search — everything
Backend	ML Service	Python + FastAPI	Image preprocessing, text detection, handwriting recognition o
Data	Primary DB	MongoDB	All collections: users, documents, ocrresults, exports, auditlogs
Data	Dev Tooling	MongoDB Compass	Visual inspection/management of the local database during de
Data	File Storage	Local disk (dev) → Cloud bucket (prod)	Uploaded images and generated export files

Why This Split (Node main backend + Python ML-only service)

Plain MERN does not include first-class support for running deep learning models like TrOCR — those libraries (PyTorch, HuggingFace Transformers) are Python-native. Rather than forcing everything into Node or everything into Python, the architecture keeps **Node.js/Express as the main backend** the React app talks to directly for all everyday operations, and isolates Python to a single internal service that Express calls only when an image needs OCR processing. This keeps the bulk of the application in familiar JavaScript while still getting proper ML tooling where it's actually required.

3. ML / AI Processing Pipeline

The AI pipeline processes uploaded images through a 6-stage inference chain, running entirely inside the Python/FastAPI service. Express calls this service over HTTP, passes it the image, and receives back the recognized text and confidence data to store in MongoDB.

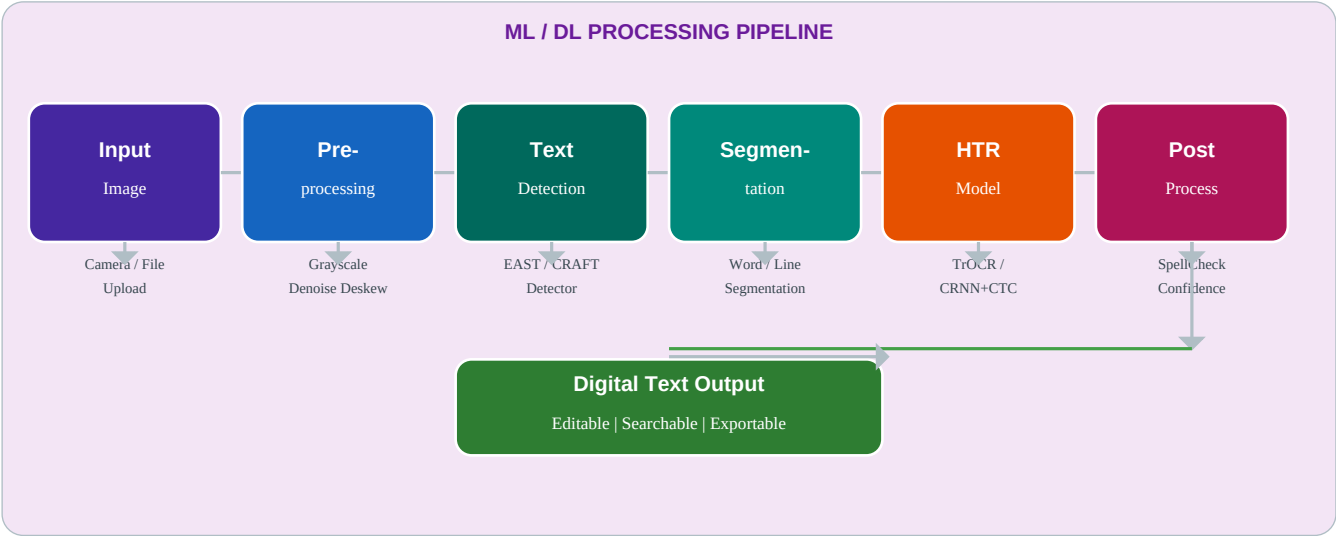


Figure 2 — ML Pipeline: From raw image input to editable text output

Stage-by-Stage Breakdown

Stage	Name	Techniques Used	Output
1	Image Input	Multi-format support (JPEG, PNG, PDF, TIFF, WebP)	Raw image / PDF pages
2	Pre-processing	Grayscale, Gaussian blur, Otsu thresholding, deskewing, morphological ops	Binary image
3	Text Detection	EAST Detector / CRAFT — finds bounding boxes of text regions	Region bounding boxes
4	Segmentation	Word-level, line-level segmentation; connected components analysis	Cropped text patches
5	HTR Model	TrOCR (microsoft/trocr-base-handwritten) + CRNN+CTC fallback	Character sequences
6	Post-processing	Spell correction (SymSpell), language model scoring, confidence scoring	Final text + confidence

Model Details

Model	Type	Task	Dataset Fine-tuned On	Accuracy
TrOCR	Transformer (ViT + GPT2)	Handwritten HTR	IAM + RIMES + custom	~96% CER
CRNN+CTC	CNN + BiLSTM + CTC	HTR fallback	IAM Handwriting DB	~91% CER
EAST	Deep Learning Detector	Text detection	ICDAR 2015	F1 ~0.87
SymSpell	Dictionary + Edit Distance	Spell correction	English Dictionary	Post-OCR fix

4. User Workflow — End-to-End

The following diagram shows how a user interacts with the system from registration to final export, including the parallel system (Node/Express) and ML engine (Python/FastAPI) activities.

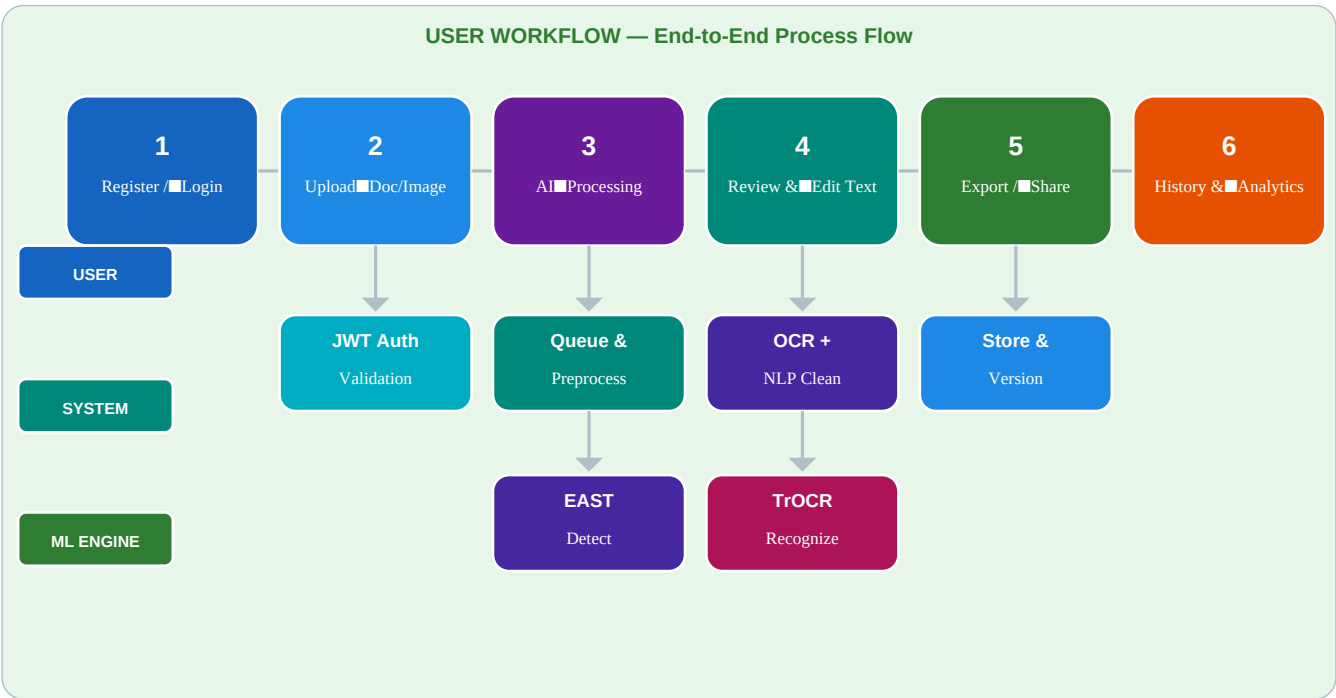


Figure 3 — User Workflow: Swimlane diagram showing User, System, and ML Engine activities

Detailed Step Descriptions

Step	Actor	Action	System Response
1 — Auth	User	Register with email + password; login	JWT access + refresh token issued by Express; bcrypt-hashed password stored
2 — Upload	User	Upload image / PDF or use browser camera capture	File validated (type/size), saved to storage, document record created
3 — Processing	ML Engine	Express calls the Python service over HTTP	EAST detects text → TrOCR transcribes → SymSpell cleans → nlpjs normalizes
4 — Review	User	View extracted text, confidence score highlights	Editable text display; low-confidence spans highlighted in red
5 — Export	User	Export as TXT (V1), DOCX / PDF (V2)	File generated by Express, stored, download link returned
6 — History	User	View past documents, search by content	MongoDB text index (\$text search) on extractedText; timeline view

5. Database Design & ERD

The database is **MongoDB**, managed locally during development with **MongoDB Compass** (so you can visually browse collections, documents, and run queries without writing scripts). Schemas are defined with **Mongoose** on the Node/Express side for validation and structure, even though MongoDB itself is schema-flexible. Full-text search over recognized text uses MongoDB's built-in **text index** (\$text operator) — no separate search engine is needed at this scale.

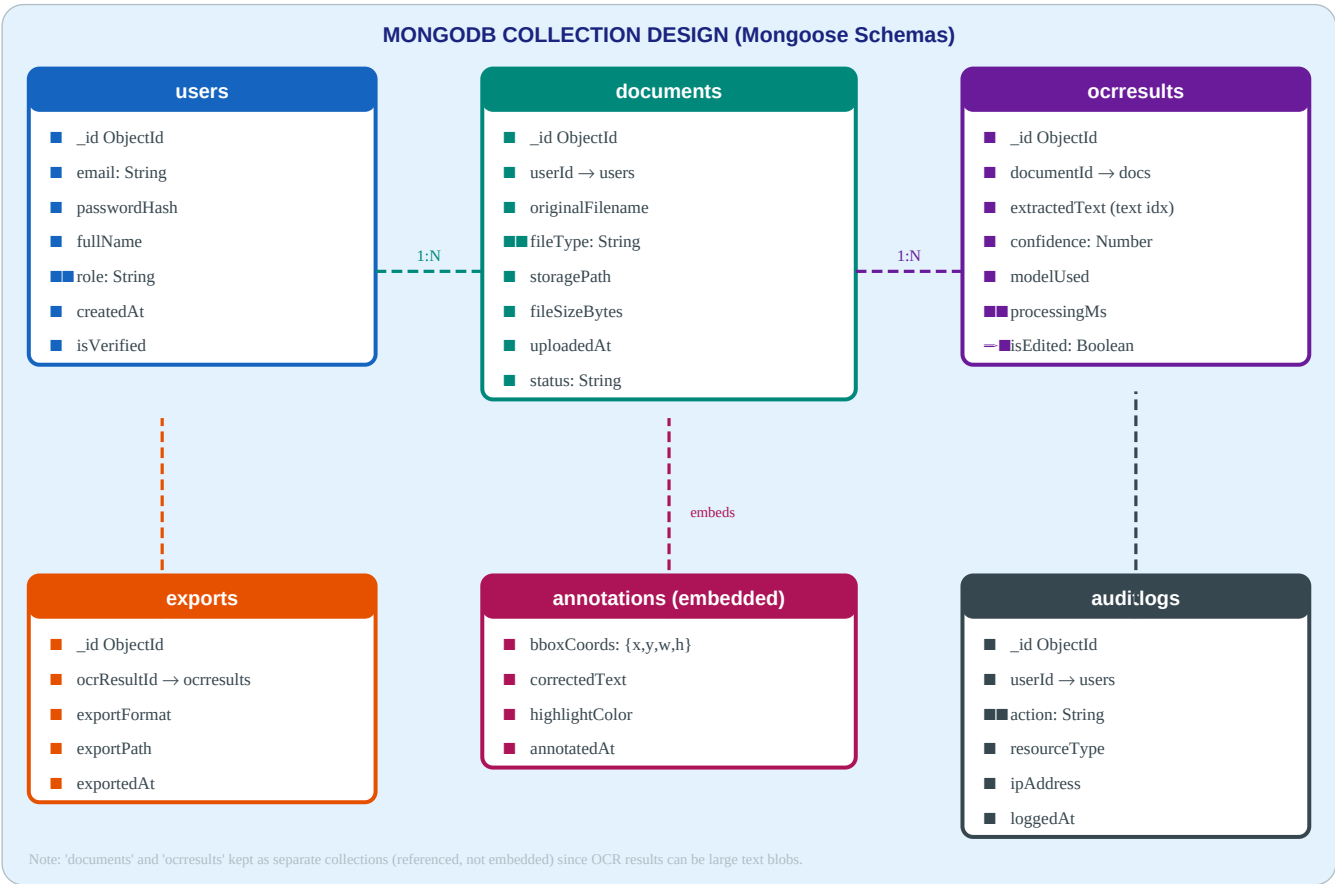


Figure 4 — Collections: users, documents, ocrresults, exports, auditlogs (annotations embedded)

Key Collection Definitions

Collection	Primary Key	References	Key Fields	Indexes
users	<code>_id</code> (ObjectId)	—	email, passwordHash, role, isVerified	email UNIQUE
documents	<code>_id</code> (ObjectId)	<code>userId</code> → users	originalFilename, fileType, storagePath, status	<code>userId</code> , status, uploadedAt
ocrresults	<code>_id</code> (ObjectId)	<code>documentId</code> → documents	extractedText, confidence, modelUsed, isEdited	text index on extractedText
exports	<code>_id</code> (ObjectId)	<code>ocrResultId</code> → ocrresults	exportFormat, exportPath, exportedAt	<code>ocrResultId</code>
auditlogs	<code>_id</code> (ObjectId)	<code>userId</code> → users	action, resourceType, ipAddress, loggedAt	<code>userId</code> , loggedAt

Mongoose Schema Snippet (Key Collections)

```
// models/User.js
const userSchema = new Schema({
  email: { type: String, required: true, unique: true, lowercase: true },
  passwordHash: { type: String, required: true },
  fullName: { type: String },
  role: { type: String, enum: ['user','admin'], default: 'user' },
  isVerified: { type: Boolean, default: false },
}, { timestamps: true });
```

```
// models/OcrResult.js
const ocrResultSchema = new Schema({
  documentId: { type: Schema.Types.ObjectId, ref: 'Document', required: true },
  extractedText: { type: String, default: '' },
  confidence: { type: Number, min: 0, max: 1 },
  modelUsed: { type: String },
  processingMs: { type: Number },
  isEdited: { type: Boolean, default: false },
  annotations: [{ bboxCoords: Object, correctedText: String, highlightColor: String }],
}, { timestamps: true });
ocrResultSchema.index({ extractedText: 'text' });
```

Using MongoDB Compass during development: connect to `mongodb://localhost:27017`, select (or create) the database (e.g. `handwriting_ocr_db`), and the collections above will appear automatically the first time each one receives a document — Mongoose creates them on first insert, no manual setup required.

6. API Endpoint Architecture

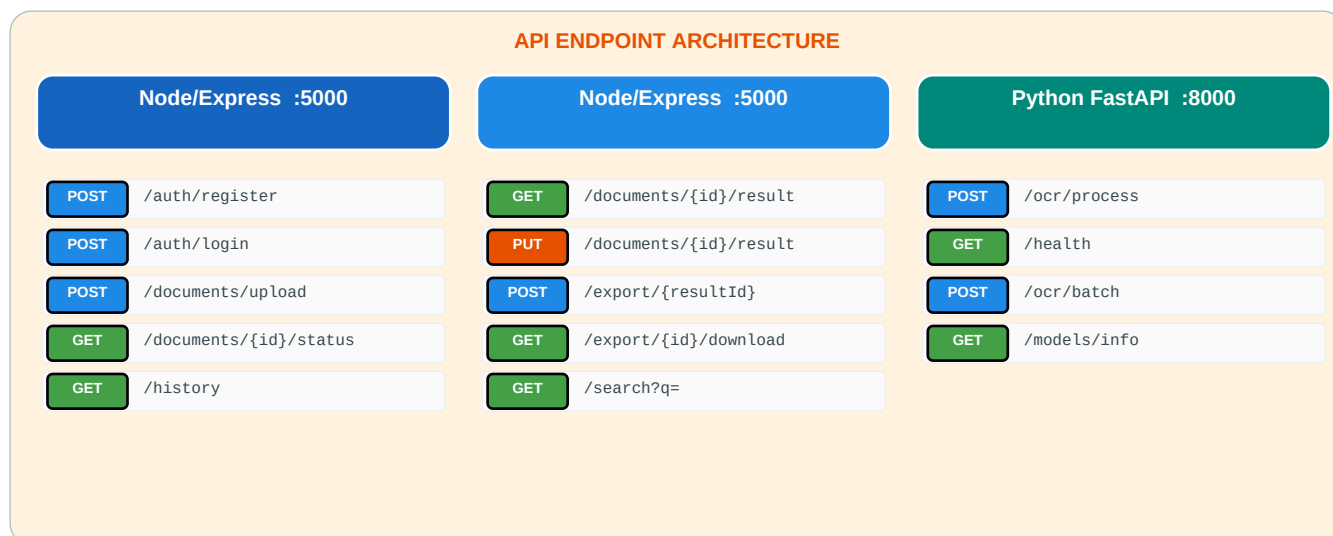


Figure 5 — REST API endpoints: Node/Express (main backend) + Python/FastAPI (OCR only)

API Contract Details

Method	Endpoint	Service	Auth	Description
POST	/auth/register	Express	None	Create new user account
POST	/auth/login	Express	None	Returns JWT access + refresh token
POST	/auth/refresh	Express	Refresh Token	Rotate JWT access token
POST	/documents/upload	Express	JWT	Upload image/PDF, returns documentId, saves to MongoDB
POST	/documents/{id}/process	Express	JWT	Calls Python service internally, triggers OCR
GET	/documents/{id}/status	Express	JWT	Poll processing status (pending/done/failed)
GET	/documents/{id}/result	Express	JWT	Fetch extracted text + confidence data
PUT	/documents/{id}/result	Express	JWT	Save user's manual edits to the recognized text
GET	/history	Express	JWT	Paginated list of user's documents
POST	/export/{resultId}	Express	JWT	Generate export (format param: txt/docx/pdf)
GET	/search	Express	JWT	Full-text search via MongoDB \$text index
POST	/ocr/process	FastAPI (internal)	Service key	Receives image, returns text + confidence — called only by Express
GET	/health	FastAPI (internal)	None	Health check for the ML service

Important: the React frontend only ever talks to the Express API. It never calls the Python service directly. Express acts as the single point of contact for the browser, and internally forwards OCR work to FastAPI. This keeps the frontend simple (one base URL to configure) and keeps the ML service unexposed to the public internet in production.

7. Technology Stack

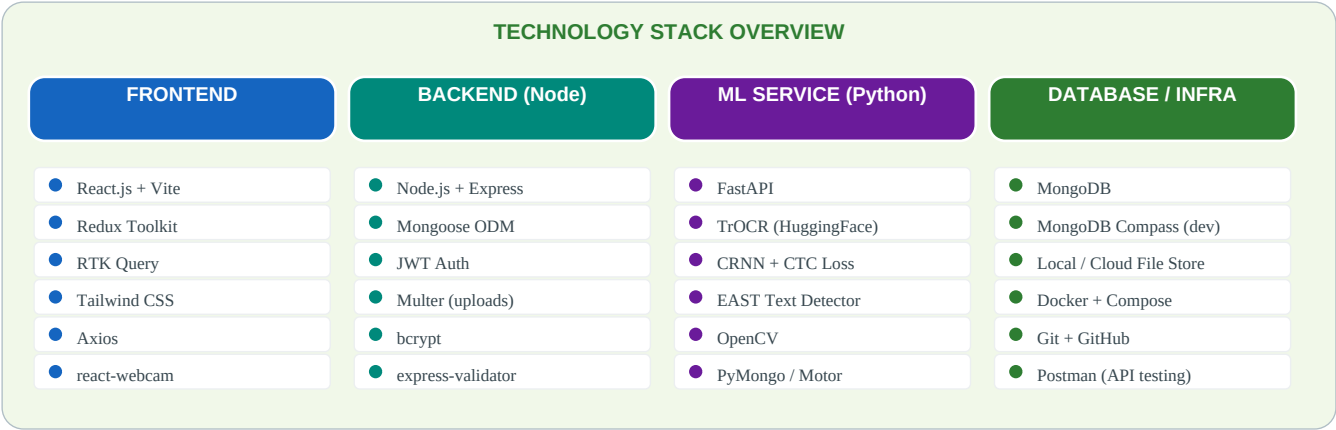


Figure 6 — Full Technology Stack across Frontend, Backend, ML Service, and Database

Why These Technologies?

Technology	Category	Why Chosen
TrOCR (HuggingFace)	AI Model	State-of-the-art handwritten OCR; pretrained on IAM; fine-tunable
Python + FastAPI	ML Service	Only place Python is used — async, auto OpenAPI docs, ideal for serving ML models
Node.js + Express	Main Backend	JavaScript across the whole stack except ML; simple, fast to build, huge ecosystem
React.js + Redux Toolkit	Frontend	Component-based, large ecosystem, RTK Query for async state and caching
Tailwind CSS	Styling	Utility-first, fast to style consistently, no separate CSS files to maintain
MongoDB	Database	JS-native document model fits MERN naturally; built-in text index covers search needs
MongoDB Compass	Dev Tooling	Visual GUI to inspect/manage the local database without writing scripts
JWT (jsonwebtoken)	Auth	Stateless auth tokens, no separate session store required for V1
Antigravity (AI coding agent)	Build Process	Used to implement the codebase from this plan; PROGRESS.md keeps work resumable across

8. Module-wise Implementation Plan

This plan is written to be handed directly to **Antigravity** (or any AI coding agent) as a build instruction. Each module should be completed, tested manually, and logged in **PROGRESS.md** before moving to the next one. If a session ends partway through a module (e.g. usage limit reached), the next session reads **PROGRESS.md**, sees exactly what was finished and what remains, and continues without re-reading the whole codebase from scratch.

Module 1: Project Setup & Folder Structure

- Initialize Git repo with three top-level folders: `frontend/`, `server/`, `ml-service/`
- Create `PROJECT_BRIEF.md` and `PROGRESS.md` at the root (instructions for the agent — see Appendix)
- Set up `frontend/` with Vite + React, install Tailwind CSS and Redux Toolkit
- Set up `server/` with Express, install Mongoose, jsonwebtoken, bcrypt, multer, dotenv
- Set up `ml-service/` with FastAPI, install uvicorn, opencv-python, transformers, torch
- Start a local MongoDB instance; verify connection through MongoDB Compass
- Add `.env` files (gitignored) for JWT secret, Mongo URI, ML service base URL

Module 2: Authentication (Node/Express)

- Define the User Mongoose schema (email, passwordHash, fullName, role, isVerified, timestamps)
- Implement bcrypt password hashing on register
- JWT token generation (access token ~15min, refresh token ~7 days)
- REST endpoints: `POST /auth/register`, `POST /auth/login`, `POST /auth/refresh`
- JWT verification middleware to protect all routes below this module

Module 3: OCR/ML Service (Python + FastAPI)

- Set up FastAPI app with a single `POST /ocr/process` endpoint accepting an image file
- OpenCV pre-processing pipeline: grayscale, denoise, deskew
- Load TrOCR model from HuggingFace; implement inference function
- Implement EAST text detector for locating text regions before recognition
- SymSpell (or equivalent) spell correction as a post-processing step
- Return JSON: { `extractedText`, `confidence`, `modelUsed`, `processingMs` }
- This service should never be called directly by the frontend — only by the Express backend

Module 4: Document Upload & OCR Orchestration (Node/Express)

- Define Document and OcrResult Mongoose schemas
- Multer-based file upload handling (type/size validation)
- `POST /documents/upload` — saves file, creates a Document record with status 'pending'
- `POST /documents/{id}/process` — Express calls the Python service, stores the result, updates status
- `GET /documents/{id}/status` and `GET /documents/{id}/result` endpoints for the frontend to poll
- `PUT /documents/{id}/result` — lets the user save manual corrections to the extracted text

Module 5: React.js Frontend

- Project structure: `app/` (Redux store), `features/` (auth, documents, ocr slices + RTK Query), `pages/`, `components/`
- Authentication pages (login/register) with RTK Query mutations, JWT stored and attached via Axios/fetch interceptor
- Upload page: drag-and-drop file upload + browser camera capture (`getUserMedia` / `react-webcam`)
- Processing/result page: polls status, then shows editable text box with low-confidence words highlighted
- History page: list of past documents with search bar (calls `GET /search`)
- Tailwind CSS used throughout for styling; no other CSS framework

Module 6: Export & Search (Node/Express)

- POST /export/{resultId} — generate .txt export in V1 (docx/pdf added in V2 using a Node library)
- GET /search — uses MongoDB \$text index on extractedText for full-text search
- GET /history — paginated list endpoint backing the frontend history page

9. Versioned Roadmap (V1 → V3)

Version	Timeline	Features	ML/AI Stack	Status
V1 — Core OCR	Month 1–2	• User auth (JWT) • Image/PDF upload + browser camera capture • TrOCR inference • Basic text export (TXT) • Web UI only • Document history	TrOCR pretrained OpenCV preprocessing EAST detector	■ Build Now
V2 — Smart Export	Month 3–4	• DOCX + searchable PDF export • Confidence highlighting UI • Annotation layer (corrections saved) • Full-text search via MongoDB • Batch upload	+ CRNN+CTC fallback + SymSpell correction + Confidence calibration	■ Planned
V3 — Advanced Features	Month 5–6	• Domain-specific fine-tuning (e.g. notes vs forms) • Multi-language support • Analytics dashboard • PWA support (installable web app, still no native app)	+ Custom fine-tuned TrOCR + Multi-language models	■ Future

Sprint Breakdown (V1 Focus — 8 Sprints × 1 week)

Antigravity should treat each sprint as one or more sessions, updating PROGRESS.md at the end of every session regardless of whether the full sprint goal was reached.

Sprint	Week	Tasks	Deliverable
S1	Wk 1	Folder setup, frontend/server/ml-service scaffolding, MongoDB connected via Compass	Running dev environment
S2	Wk 2	Express auth: User model, JWT, /auth/register, /auth/login, /auth/refresh	Auth API live
S3	Wk 3	FastAPI setup, OpenCV preprocessing, TrOCR integration, /ocr/process endpoint	OCR inference working
S4	Wk 4	Document upload (Multer), Document + OcrResult models, Express ↔ FastAPI integration	End-to-end OCR pipeline working
S5	Wk 5	React setup, auth flows, file upload + camera capture UI, Redux store	Frontend skeleton live
S6	Wk 6	Text editor with confidence highlights, history page with search	Full V1 UI working
S7	Wk 7	TXT export, MongoDB text index search, manual testing pass	V1 feature-complete
S8	Wk 8	Bug fixes, README, final PROGRESS.md update marking V1 done	V1 release ready

10. Deployment & DevOps Pipeline

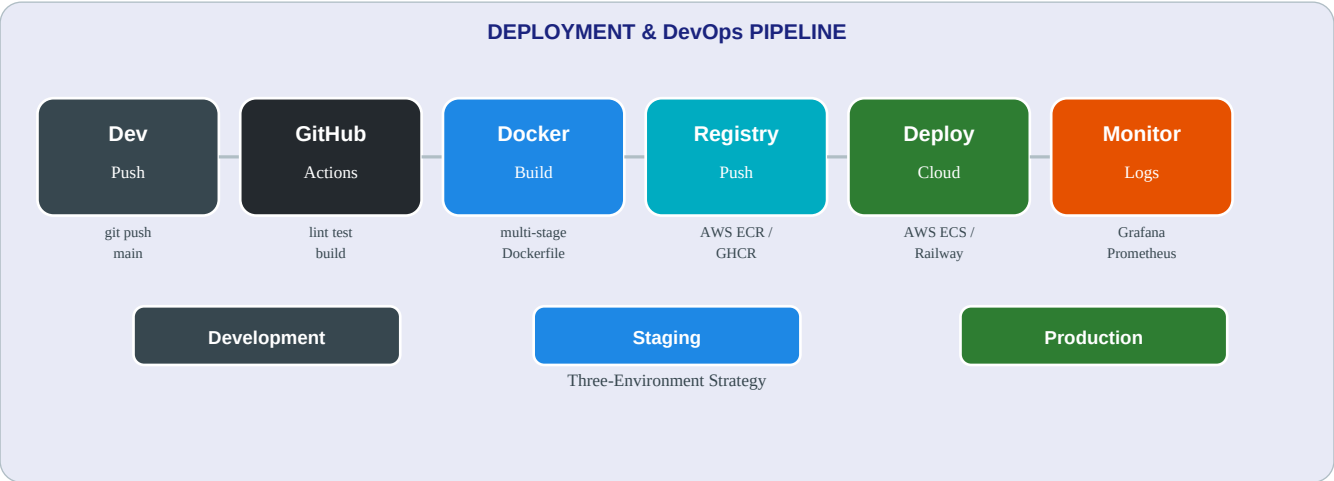


Figure 7 — DevOps Pipeline: Code push to production across three environments

Docker Compose Configuration (Simplified)

```
version: '3.9'
services:
  client: build: ./frontend | port: 5173 (dev) | served as static build in prod
  server: build: ./server | port: 5000 | depends: mongo
  ml-service: build: ./ml-service | port: 8000 | depends: mongo
  mongo: image: mongo:7 | port: 27017 | volume: mongodata
```

Docker Compose is optional for this project's scale — during development it's usually simpler to just run `npm run dev` in `frontend/`, `npm run dev` in `server/`, and `uvicorn app.main:app --reload` in `ml-service/`, alongside a locally installed MongoDB that Compass connects to. Docker becomes more useful once it's time to deploy.

Service	Platform Options	Deployment Method	Notes
React (frontend)	Vercel / Netlify	Static build, CDN deploy	Free tier is sufficient for V1
Node/Express (server)	Railway / Render	Docker container or direct Node deploy	Set env vars: MONGO_URI, JWT_SECRET, ML_SERVICE_U
Python/FastAPI (ml-service)	Railway / Render	Docker container, may need more RAM/CPU	Keep this service private — not exposed to the public internet
MongoDB	MongoDB Atlas (free tier)	Managed service	Local Compass connects to Atlas the same way it connects lo
File Storage	Local disk (dev) → Cloudinary / S3 (prod)	SDS integration (prod)	Swap storage logic behind one small module so it's a one-plac

11. Security Architecture

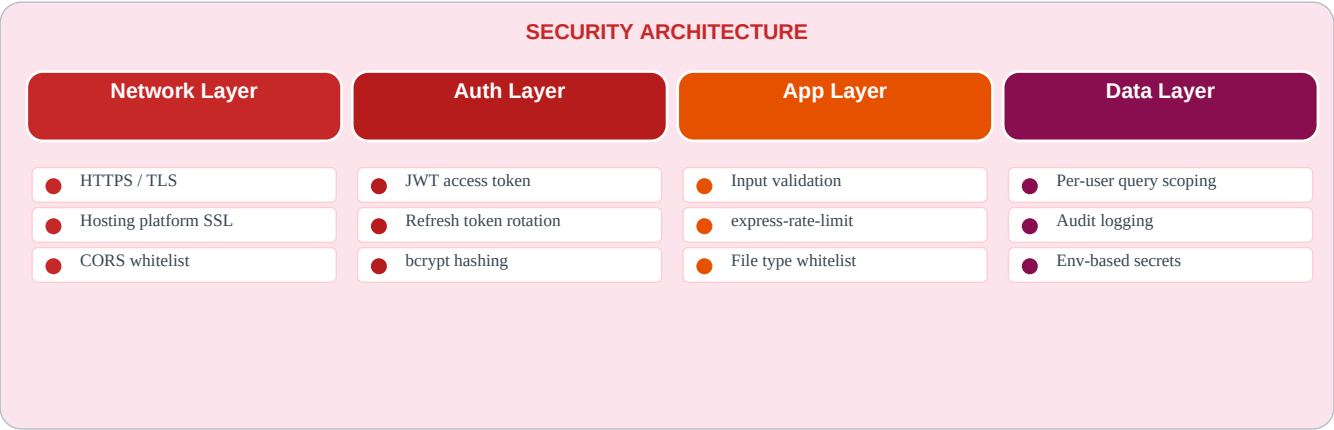


Figure 8 — Security layers from network to data storage

Security Layer	Threat Mitigated	Implementation
HTTPS / TLS	Man-in-the-Middle	Enforced by hosting platform (Vercel/Railway provide this automatically)
JWT + Refresh Rotation	Session hijacking	Short-lived access tokens (~15min), refresh token stored in httpOnly cookie
Rate Limiting	Brute force, abuse	express-rate-limit middleware on auth and upload routes
Input Validation	Injection attacks	express-validator on Node routes, Pydantic models on FastAPI routes, file type whitelist
Password Hashing	Credential theft	bcrypt with salt rounds ≥ 10 on the User model
MongoDB Access Control	Data leakage	Application-level checks: every query filters by the requesting user's userId
Audit Logging	Non-repudiation	All mutations logged to the auditlogs collection with IP, timestamp, action
CORS Policy	Cross-site attacks	Express CORS middleware whitelisting only the deployed frontend domain

12. Testing Strategy

Test Type	Tool	What is Tested	Target Coverage
Unit Tests	Jest (Node), pytest (FastAPI)	Auth logic, Mongoose model validation, OCR helper functions, business logic functions	Key business logic functions
Integration Tests	Supertest (Node), pytest + TestClient (FastAPI)	API endpoints, DB writes/reads, file upload	All endpoints listed in Section 6
E2E Tests	Playwright	Full user flows: register → upload → review → export → delete	3-5 main user flows
Manual Testing	Postman / Thunder Client	Every endpoint hit manually at least once before marking as done	Every endpoint done
Model Accuracy	Custom eval script	Character Error Rate (CER) on a small held-out sample of handwriting images	CER < 10% using ViT base
Security Spot-Checks	Manual review + OWASP ZAP (optional)	Auth bypass attempts, input validation, exposed secrets	No critical issues before deployment

13. Risks & Mitigations

Risk	Impact	Probability	Mitigation Strategy
Low OCR accuracy on highly stylized handwriting	High	Medium	Fine-tune TrOCR on domain-specific data; CRNN as fallback; user correction feedback loop
Slow inference on CPU (no GPU available)	Medium	High	Use a smaller/quantized TrOCR variant for V1; cache results for identical re-uploaded images
MongoDB Atlas free tier limits in production	Medium	Medium	Keep document/text sizes reasonable; archive old documents; upgrade tier only when actually needed
Browser camera capture inconsistency across devices	Medium	High	Client-side image enhancement (brightness/contrast) before sending to the server; clear capture instructions
JWT token compromise	High	Low	Short expiry (~15min) on access tokens, refresh token rotation, logout invalidates refresh token
Antigravity session ends mid-task / message limit	Medium	Medium	PROGRESS.md updated after every session is mandatory — a new session (even on a different device) can resume
Model drift / accuracy degrading over time	Medium	Low	Periodic manual accuracy spot-checks against a small held-out test set of sample handwriting

14. Building This With Antigravity (AI Coding Agent)

This project is meant to be implemented by **Antigravity**, an AI coding agent, working directly from two reference files kept at the root of the repository. This section explains how that process works and why it's set up this way.

The Two Reference Files

File	Purpose
PROJECT_BRIEF.md	The permanent reference: tech stack, folder structure, feature scope, and rules the agent must always follow. This ra
PROGRESS.md	The living work log: updated at the end of every session with what was done, what works, what's broken, and exactly

Why This Matters: Session Continuity

AI coding agents like Antigravity work in sessions, and a session can end before a feature is fully complete — due to a usage limit being reached, a context window filling up, or simply needing to stop for the day. Without a structured handoff, the next session (possibly under a different account) would have to re-read the entire codebase to figure out what's done and what isn't, which wastes time and risks redoing or conflicting with previous work. The PROGRESS.md file solves this: it is the single place where current status, what just changed, and what to do next are always recorded in plain language.

The Workflow, Step by Step

1. Open a new Antigravity session and point it at the project folder.
2. Antigravity reads PROJECT_BRIEF.md first — this re-establishes the tech stack and rules, so it never reintroduces dropped technologies (no Flutter, no SQL databases, etc.).
3. Antigravity reads PROGRESS.md next — this tells it exactly what's already built, what's in progress, and the next task.
4. Antigravity does the work for that session: writes code, tests it manually, fixes issues.
5. Before ending the session, Antigravity appends a new entry to PROGRESS.md following the template inside that file — what was done, what changed, what's next, and any gotchas.
6. If the session is cut short (e.g. usage limit hit) before a clean stopping point, Antigravity should still update PROGRESS.md with whatever is true at that moment, even if incomplete — partial honesty beats silence.
7. The next session (same account or a different one) repeats from step 1, picking up exactly where the last one left off.

What Goes in Each PROGRESS.md Entry

Worked on by · Goal for this session · What was actually done · Files created/changed · What works now that didn't before · What's still broken or incomplete · Decisions made (and why) · Next session should do · Notes for whoever continues this.

This level of detail might feel excessive for a small project, but it costs very little time to write and saves a large amount of time and confusion the moment more than one session — or more than one person — is involved in building the project.